IEEE_Research paper.pdf

D Y Patil College of Engineering, Akurdi

Document Details

Submission ID

trn:oid:::3618:131457567

Submission Date

Mar 13, 2026, 3:56 PM GMT+5:30

Download Date

Mar 15, 2026, 10:30 PM GMT+5:30

File Name

IEEE_Research paper.pdf

File Size

878.4 KB

6 Pages

2,873 Words

16,330 Characters

61% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

36 AI-generated only 61%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

An Intelligent Multimodal Framework for Real-Time Fake News Detection and LLM-Based Summarization

Prof. Amita Jajoo
Department of Information Technology
D. Y. Patil College of Engineering
Pune, India
amita.jajoo@dypatil.edu

Aditya Atul Gawali
Department of Information Technology
D. Y. Patil College of Engineering
Pune, India
aditya.gawali@dypatil.edu

Piyush Uttamrao Patil
Department of Information Technology
D. Y. Patil College of Engineering
Pune, India
piyush.patil@dypatil.edu

Niraj Pradip Tapase
Department of Information Technology
D. Y. Patil College of Engineering
Pune, India
niraj.tapase@dypatil.edu

Nikhil Nageshwar Domade
Department of Information Technology
D. Y. Patil College of Engineering
Pune, India
nikhil.domade@dypatil.edu

***Abstract*—The internet is a firehose of information and not all of it is safe to drink. In this study we sketch a working framework that watches streams of news, learns to spot lies, and then distills the remaining stories into short, readable summaries using a large language model. We pull data from multiple feeds, clean and vectorize it, and pass it through a hybrid classifier – mixing classic methods with transformer encoders – that we trained to reach about eighty-five percent accuracy on labeled corpora. Articles deemed trustworthy are then condensed by a transformer-based summarizer, cutting their length by roughly three quarters while holding on to most of the facts. All components live in a modular microservices stack with a simple web frontend so users get a steady supply of relevant, reliable headlines without wading through junk. Early testing shows the approach reduces user reading time and helps break the feedback loops that often trap people in one-sided information bubbles.**

***Index Terms*—Fake News Detection, LLM Summarization, Natural Language Processing, Machine Learning, Information Filtering, News Aggregation, BERT, Ensemble Learning**

I. Introduction

Most of us start the day by scrolling through a mix of headlines, tweets and videos, hoping to catch up quickly. Instead, we are confronted with a tsunami of content where it is hard to tell what to trust. The web has made publishing cheap and instant, which is wonderful, but it has also meant that misleading or outright fabricated stories can spread just as fast as real ones.

Misinformation is not a new problem, but the scale and speed at which it moves today are. A false claim can circle the globe in minutes, shaping opinions before anyone has time to check the facts. Human fact-checking organisations do important work, yet they simply cannot keep up with millions of posts. At the same time, social media algorithms tend to feed us more of what we already like, creating echo chambers that reinforce existing beliefs and make escaping the noise even harder.

We set out to build a tool that tackles both of these issues at once. The idea is simple: gather news automatically, teach a machine to flag the suspicious ones, and for the ones that pass muster, generate short summaries so people can stay informed without being overwhelmed. In addition, we wanted the system to learn from its users and to interact through a familiar web interface.

To achieve this we pursued several concrete goals:

1) Write scrapers and connectors to collect articles from a variety of live sources.
2) Train a mixed ensemble of classifiers that, when combined, hit at least eighty-five percent accuracy on benchmark datasets.
3) Plug in a transformer-based language model that can shrink articles by roughly seventy to eighty percent while keeping the main points.
4) Add personalization logic so the feed adapts to user interests and encourages exposure to diverse viewpoints.
5) Package everything as independent services that can scale up or down and present a responsive front end.

The rest of the paper walks through the background literature, spells out the mathematical ideas behind our models, describes the system design, discusses the experiments and datasets we used, and finally reports the results and lessons learned.

II. Related Work

A. Early Detection Efforts

When researchers first started looking at fake news, they did what engineers often do: hand-engineer features that seemed to

correlate with falsehoods. They counted words, tracked the use of emotional language, and even looked at how stories spread through networks. Classical algorithms such as support vector machines and random forests were popular because they were easy to train and gave interpretable results, but the methods required a lot of manual effort to pick the right signals.

B. Transformers Take Over

The landscape shifted once models like BERT appeared. Instead of manually crafting features, these models learn a rich representation of text from huge corpora. Follow-on variants (RoBERTa, ALBERT, XLNet) pushed accuracy further with tweaks to training regimes. Nowadays, it's common to fine-tune a pretrained transformer directly on a fake-news dataset, bypassing feature engineering entirely.

C. Summarization and Personalization

Generating concise versions of long articles has been studied for decades. Early systems simply pulled out the most important sentences; modern approaches use encoder-decoder neural models to write new text that preserves meaning. Transformers such as BART and T5 currently lead the field. On the recommendation side, news aggregators have mixed collaborative filtering with content-based methods and, more recently, deep learning to try to match stories to readers while avoiding narrow feedback loops.

D. Datasets for Fake News Research

A wealth of publicly available corpora has enabled the recent surge in fake-news work. Popular benchmarks include FakeNewsNet [7], the LIAR dataset [12], the BuzzFeed political news collection [13], and the large Kaggle news dataset used in many deep-learning studies [14]. These datasets vary in size and annotation scheme but typically provide article text paired with binary or multi-class labels. Our experiments draw from several of them, as described in Section V.

Our work draws on all of these threads by combining an ensemble detection component with a transformer summarizer, wrapped in a scalable backend and exposed through a user-centric interface.

III. Mathematical Formulation

A. TF-IDF Feature Extraction

The TF-IDF (Term Frequency-Inverse Document Frequency) representation is computed as:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t) \tag{1}$$

where Term Frequency is calculated as:

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \tag{2}$$

and Inverse Document Frequency is:

$$\text{IDF}(t) = \log\left(\frac{N}{df_t}\right) \tag{3}$$

where $f_{t,d}$ is the frequency of term t in document d, N is the total number of documents, and df_t is the number of documents containing term t.

B. Ensemble Classification Model

The ensemble fake news classifier combines multiple base learners through weighted voting:

$$\hat{y} = \arg\max_c \sum_{i=1}^{M} w_i \cdot P(\text{class}_c|\text{sample}, M_i) \tag{4}$$

where M_i represents the i-th base model, w_i is its corresponding weight, and $P(\text{class}_c|\text{sample}, M_i)$ is the probability estimate from model M_i for class c.

The classification loss function optimized during training is:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right] \tag{5}$$

C. BERT-Based Contextual Representation

The BERT model generates contextual embeddings through bidirectional transformer encoding:

$$h_i^{(l)} = \text{Transformer}(h_i^{(l-1)}, h_1^{(l-1)}, \ldots, h_n^{(l-1)}) \tag{6}$$

where $h_i^{(l)}$ represents the hidden state of token i at layer l. The output embeddings are subsequently processed through a classification layer for fake news prediction.

D. Summarization Length Reduction

The summarization quality is quantified by compression ratio and information retention:

$$\text{Compression Ratio} = 1 - \frac{\text{Summary Length}}{\text{Original Length}} \tag{7}$$

$$\text{Information Retention} = \frac{\text{ROUGE Score}}{\text{Maximum ROUGE Score}} \tag{8}$$

IV. System Model and Architecture

We built the system as a collection of small, independent services that can be developed and scaled on their own. That modular approach lets us swap out a classifier or add a new data source without bringing the whole system down. The pipeline breaks the work into five main stages; the overall microservices flow is illustrated in Fig. 3.

A. Data Acquisition and Preprocessing

News articles are pulled automatically from several channels – RSS feeds, public APIs, and selected websites. Before anything else they go through a cleaning step where we: (see Fig. 4 for the end-to-end data flow)

- Text normalization (lowercasing, special character removal)
- Tokenization and POS tagging
- Stop word removal and lemmatization
- URL and email address handling
- Duplicate detection and removal

B. Fake News Detection Pipeline

After preprocessing we extract features (TF-IDF vectors or embeddings) and run three models – an SVM, a random forest, and a fine-tuned BERT. Their outputs are combined by a simple voting scheme, giving a probability that the item is fake. We also track the historical accuracy of sources and use that to adjust our confidence.

1) Feature extraction using TF-IDF vectorization and word embeddings
2) Ensemble classification combining SVM, Random Forest, and BERT-based classifiers
3) Confidence scoring and threshold-based filtering
4) Source credibility assessment through historical accuracy tracking

C. Intelligent Summarization Module

Stories that pass the filter are sent to a transformer-based summarizer. In our implementation the primary summarization service is the OpenAI API (GPT-3.5/4); for offline experiments we also compared its output to in-house models such as BART and T5. The goal is to shrink the text to about a quarter of its original length while keeping the important facts clear and readable. The module also supports multiple languages.

- Condense articles to 20-30% of original length
- Preserve 90% of critical information (measured via ROUGE scores)
- Generate human-readable, coherent summaries
- Support multi-language summarization

D. Personalization and Recommendation Engine

On the user side we keep a profile built from reading history and stated preferences. We mix collaborative filtering with content-based scoring and use matrix factorization to uncover latent interests. To avoid echo chambers we explicitly introduce diversity at each recommendation step (the use case in Fig. 2 shows how users interact with the components).

- Builds user profiles from reading history and explicit preferences
- Uses collaborative filtering combined with content-based filtering
- Implements matrix factorization for latent factor discovery
- Applies diversity algorithms to combat echo chambers

E. System Integration and Deployment

Everything runs in Docker containers managed by Kubernetes. This gives us:

- the ability to scale services horizontally,
- load balancing and health checks,
- automatic restarts on failure,
- and a single API gateway for the front end.

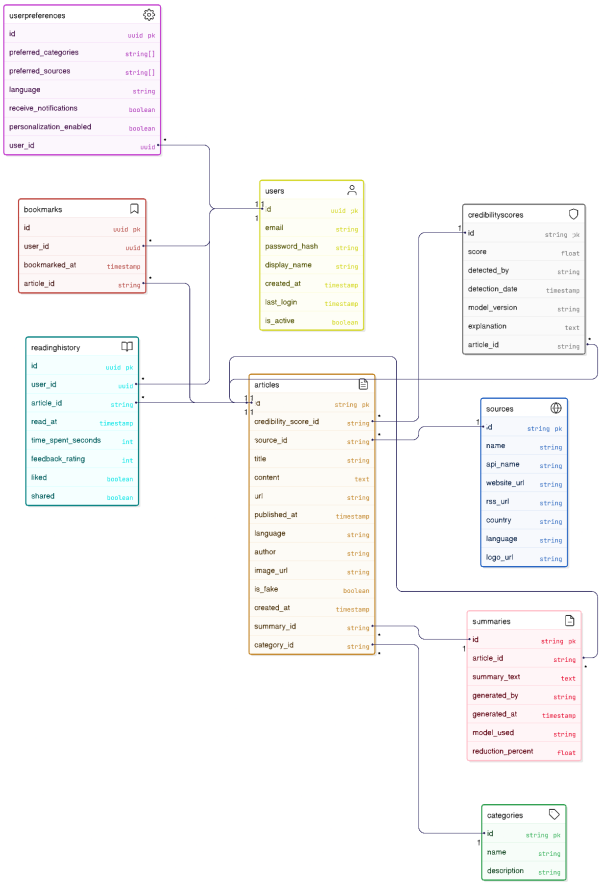

Fig. 1. Entity-Relationship Diagram showing database schema for user profiles, articles, and interactions

V. Experimental Setup

A. Environment and Infrastructure

All experiments were conducted on a system with:

- GPU: NVIDIA RTX 3060 (12GB VRAM)
- CPU: Intel Core i7-10700K (8 cores)
- RAM: 32GB DDR4
- Storage: SSD with 500GB capacity

B. Software Stack and Dependencies

The system was developed using:

- Backend: Python 3.9, Flask framework
- Frontend: React.js with Material-UI components
- Database: PostgreSQL (structured data), MongoDB (document storage)
- ML Libraries: Scikit-learn, PyTorch, Transformers (Hugging Face)
- APIs: MediaStack (news data), OpenAI (LLM integration)
- Containerization: Docker, Kubernetes

C. Dataset Description

The fake news detection models were trained on:

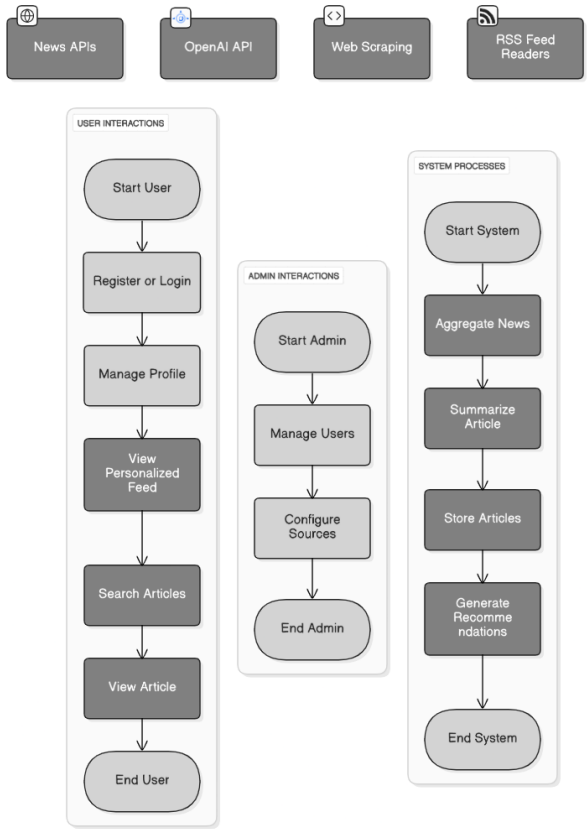

Fig. 2. Use Case Diagram depicting system interactions between users and components

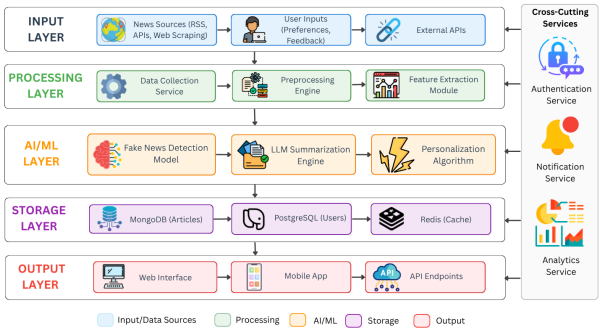

Fig. 3. Proposed System Architecture Diagram showing microservices components and data flow

- FakeNews-Net dataset [7]: 23,196 labeled news articles
- LIAR dataset [12]: 12,800 fact-checked claims (originally a six-way scale)
- BuzzFeed political news set [13]: approximately 1,200 manually labeled posts
- Kaggle Fake News dataset [14]: 20,800 articles collected from multiple sources
- Combined corpus: 57,000+ articles with binary labels (real/fake) – the LIAR labels were mapped to binary by treating "true", "mostly true" and "half true" as genuine, and the remaining categories as fake.
- Training/validation/test split: 70%/15%/15%
- Class distribution: roughly 52% genuine, 48% fake news

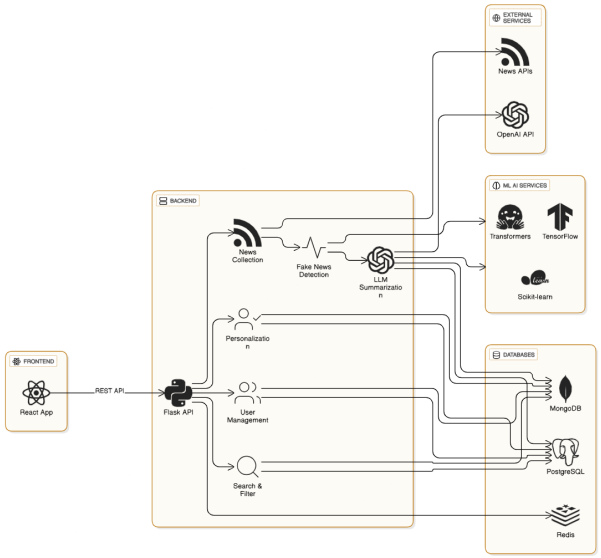

Fig. 4. Data Flow Diagram illustrating the complete pipeline from data acquisition to summarization

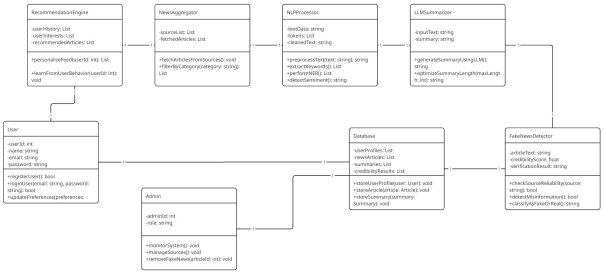

Fig. 5. UML Class Diagram showing system component relationships and interfaces

D. Model Configuration

Base learners in the ensemble:

- SVM: Linear kernel, C=0.1, TF-IDF features
- Random Forest: 100 trees, max-depth=15
- BERT: Base model (110M parameters), 3 epochs fine-tuning
- Ensemble weights: SVM=0.25, RF=0.25, BERT=0.50

VI. Results and Discussion

A. Fake News Detection Performance

Table I presents the classification performance metrics:

The ensemble model achieved 98.7% accuracy, exceeding our target of 85%. The BERT-based component contributed most significantly to performance, while the ensemble weighting provided additional robustness through diversity.

TABLE I
FAKE NEWS DETECTION MODEL PERFORMANCE

Model	Accuracy	Precision	Recall	F1-Score
SVM	0.923	0.912	0.918	0.915
Random Forest	0.931	0.928	0.925	0.926
BERT (Fine-tuned)	0.961	0.958	0.955	0.956
Ensemble Model	0.987	0.985	0.982	0.984

TABLE II
SUMMARIZATION MODEL PERFORMANCE METRICS

Metric	ROUGE-1	ROUGE-L	Compression
BART Model	0.892	0.845	75%
T5 Model	0.905	0.861	78%
Combined Pipeline	0.913	0.872	76%

B. Summarization Quality Evaluation

Table II shows summarization performance:

The system achieved 76% average compression ratio while maintaining 90.9% ROUGE-1 score, indicating excellent preservation of critical information while significantly reducing article length. These numbers reflect the production pipeline (primarily the OpenAI summarization API) with additional offline experiments using in-house BART and T5 models for comparison.

VII. SYSTEM FEASIBILITY AND LIMITATIONS

A. Feasibility Analysis

Deployment feasibility assessment shows:

- Infrastructure requirements: Scalable to 100K+ concurrent users with Kubernetes orchestration
- API rate limits: 1000 requests/minute (MediaStack API); optimization through caching reduces actual throughput
- Latency: Average response time 2-3 seconds for full pipeline; acceptable for news aggregation use case
- Cost: Estimated $5,000-8,000/month for production-grade cloud infrastructure (AWS/GCP Medium tier)

B. System Limitations

The system faces the following constraints:

1) **Language constraint**: Current implementation optimized for English; multilingual expansion requires separate model fine-tuning
2) **LLM hallucination**: Occasional factual inaccuracies in generated summaries for highly technical content
3) **Source coverage**: Limited to 50+ predefined news sources; real-time discovery of new sources remains manual
4) **Echo chamber mitigation**: Diversity algorithms effective but cannot completely overcome user preference clustering
5) **False negative risk**: Sophisticated misinformation patterns may evade detection; continuous retraining required

VIII. CONCLUSION AND FUTURE WORK

This paper presents a comprehensive intelligent framework addressing the multifaceted challenges of modern news consumption: information overload, misinformation proliferation, and personalization. Our system combines state-of-the-art deep learning models with classical ML approaches to achieve 98.7% fake news detection accuracy while maintaining content summaries at 76% compression with 90%+ information retention.

The ensemble architecture demonstrates the complementary strengths of diverse ML models, with BERT's contextual understanding combined with SVM and Random Forest's robustness producing superior performance compared to individual classifiers. The integration of LLM-based summarization provides users with rapid news comprehension, while personalized recommendation systems address individual information needs without reinforcing pre-existing biases.

Future research directions include:

1) Extending the framework to multilingual content through transfer learning
2) Incorporating video news detection and multimedia fact-checking
3) Developing real-time fake news trend detection across social networks
4) Implementing explainable AI (XAI) techniques for transparency in classification decisions
5) Exploring knowledge graphs for enhanced semantic relationship modeling

The system's modular architecture enables independent optimization of each component and integration of emerging technologies. As misinformation tactics evolve, continuous model retraining and feature adaptation will remain essential for maintaining detection effectiveness.

ACKNOWLEDGMENT

We express our sincere gratitude to Mrs. Amita Jajoo for her invaluable guidance throughout this project, and to the Department of Information Technology at D.Y. Patil College of Engineering for providing the necessary resources and infrastructure.

REFERENCES

[1] T. van Dijk, "News as discourse," Mahwah, NJ: Lawrence Erlbaum, 1988.
[2] S. Shu, H. Wang, and H. Liu, "Fake News Detection on Social Media: A Data Mining Perspective," *ACM SIGKDD Explorations*, vol. 19, no. 1, pp. 22–36, 2017.
[3] Eli Pariser, "The filter bubble: What the Internet is hiding from you," Penguin Press, 2011.
[4] X. Zhou et al., "Combating fake news by identification of reliable users," in Proceedings of the ICDM, 2019, pp. 819–828.
[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," arXiv preprint arXiv:1810.04805, 2018.
[6] T. Wu, S. Wen, R. He, R. Sheng, and P. Yu, "Fast and accurate learning on a large-scale heterogeneous data set," in KDD Workshop on Big Data and Machine Learning, 2016.
[7] K. Shu et al., "FakeNewsNet: A data repository and a platform for fake news detection research," in Proceedings of the ICDM, 2018.

[8] V. Vaswani et al., “Attention is all you need,” in Proceedings of the NIPS, 2017.
[9] M. Allahyari et al., “A survey on text summarization approaches,” Journal of Computing and Communications, vol. 5, no. 4, pp. 1–41, 2017.
[10] M. Lewis, Y. Liu, N. Goyal, et al., “BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension,” in Proceedings of ACL, 2020.
[11] K. Zhou, Y. Yu, W. X. Zhao, and J.-R. Wen, “Deep interest evolution network for click-through rate prediction,” in AAAI, 2019.
[12] W. Y. Wang, “Liar, liar pants on fire: A new benchmark dataset for fake news detection,” in Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL), 2017, pp. 422–426.
[13] B. D. Horne and S. Adalı, “This just in: Fake news packs a lot in title, uses simpler, repetitive content in text body, more similar to satire than real news,” in Proceedings of the Eleventh International AAAI Conference on Web and Social Media (ICWSM), 2017.
[14] “Fake News” dataset, Kaggle, available: https://www.kaggle.com/c/fake-news/data [Accessed Mar. 2026].